

Seed stability of item-restscore decisions

B = 400 against B = 1000 for RMitemRestscoreCutoff()

Magnus Johansson

2026-10-03

Table of contents

Purpose	1
Design	1
Data	2
One run	3
Run	5
Results	6
Two-analyst disagreement	6
Unstable items	8
Flags per dataset	9
Interpretation	10

Purpose

The bootstrap item-restscore test holds its family-wise rate at $B = 400$ (`restscore_cutoff_validation.qmd`). The number of iterations still matters for two things that the error rate does not show:

1. **Seed dependence.** Two analysts who run the same data with different seeds can reach different decisions about an item near the boundary. For infit this happened for at least one item in about 10 percent of analyses at $B = 400$ and 4 percent at $B = 2000$ (Johansson, 2026). Restscore has not been measured.
2. **BH resolution.** With k items, BH flags anything only if the smallest p-value is at most α/k . At $B = 400$ the smallest attainable p-value is $1/401 = .0025$, which is α/k for $k = 20$, so BH with many items depends on the observed value lying beyond every simulated one.

This study measures both at $B = 400$ and $B = 1000$ with the conditional DGP, the package default.

Candidate tests. `restscore_eti_null.qmd` and `restscore_power_conditional.qmd` tested an equal-tailed alternative to the shipped `|t|`: two one-sided Westfall-Young step-downs, one per direction at $\alpha/2$, with the mean and SD computed from all $B + 1$ values (`split_pooled`). Its critical value is roughly the 10th largest of 400 simulated maxima instead of the 20th, so it may depend more on the seed. It is computed here from the same cutoff simulation, together with `|t|` with pooled moments (`wy_pooled`), so seed stability can be compared at no extra run time. Both are Westfall-Young only.

Design

Datasets. Five cells, 20 datasets each, regenerated from the seeds of the earlier studies so the data are the same:

cell	source	why
<code>underfit_litem_t0_n400</code>	power study	one underfitting item, near the boundary at this n

cell	source	why
underfit_2item_t0_n400	power study	two items masking each other, many borderline decisions
overfit_1item_t0_n150	power study	overfit at small n
overfit_1item_tm2_n400	power study	overfit off target
dich_tgt_300	validation study	null, 20 dichotomous items, where BH resolution binds

Seeds. Five cutoff seeds per dataset at each of $B = 400$ and $B = 1000$.

Outcomes, separately for WY, BH and the two candidates:

- **Two-analyst disagreement:** over the 10 pairs of seeds per dataset, the share of pairs that disagree on the flag of at least one item.
- **Unstable items:** the share of items whose flag (none, overfit or underfit) is not the same in all five seeds.
- **Flags per dataset**, averaged over seeds, to show whether B shifts decisions in one direction.

Cost. About 0.1 s per iteration, so 7,000 iterations per dataset and about 100 datasets, roughly 20 CPU-hours or 2 hours on 10 cores. Results are saved per cell, so an interrupted render resumes. Set `R_DATASETS` lower for a quick run.

```
suppressMessages({
  devtools::load_all("...", quiet = TRUE)
  library(parallel)
  library(dplyr)
  library(tidyr)
})
options(rgl.useNULL = TRUE)

stopifnot(
  "conditional is not the RMitemRestscoreCutoff() default" =
    identical(eval(formals(RMitemRestscoreCutoff)$dgp)[1], "conditional")
)

R_DATASETS <- 20
N_SEEDS    <- 5
BS         <- c(400, 1000)
NCORES     <- 10
RES_DIR    <- "restscore_stability"
dir.create(RES_DIR, showWarnings = FALSE)

source("restscore_power_generator.R")
RHO <- 0.85
SLOPE <- 1.6
```

Data

The power-study seed is $1e6 * (\text{row in its 32-cell grid}) + \text{rep}$, and the validation-study seed is $1e6 * (\text{row in its 4-cell grid}) + \text{rep}$, so both grids are rebuilt in their original order.

```
power_grid <- expand.grid(
  n = c(150, 250, 400, 600),
  target = c(0, -2),
```

```

n_misfit = c(1, 2),
direction = c("underfit", "overfit"),
stringsAsFactors = FALSE
)
power_grid$cell <- with(power_grid, sprintf(
  "%s_%ditem_t%s_n%d", direction, n_misfit, ifelse(target == 0, "0", "m2"), n
))

val_cells <- c("dich_mis_150", "dich_tgt_300", "poly_tgt_150", "poly_tgt_1000")
sim_pcm <- function(theta, thr) {
  n <- length(theta)
  sapply(thr, function(d) {
    m <- length(d)
    eta <- outer(theta, 0:m) -
      matrix(c(0, cumsum(d)), n, m + 1, byrow = TRUE)
    p <- exp(eta - apply(eta, 1, max))
    p <- p / rowSums(p)
    cp <- p %%% upper.tri(diag(m + 1), diag = TRUE)
    rowSums(runif(n) > cp)
  })
}

cells <- c("underfit_litem_t0_n400", "underfit_2item_t0_n400",
  "overfit_litem_t0_n150", "overfit_litem_tm2_n400", "dich_tgt_300")

make_data <- function(cell, rep) {
  if (cell == "dich_tgt_300") {
    thr <- as.list(seq(-2, 2, length.out = 20))
    set.seed(1e6 * match(cell, val_cells) + rep)
    repeat {
      d <- as.data.frame(sim_pcm(rnorm(300, 0, 1.5), thr))
      names(d) <- paste0("I", seq_along(thr))
      if (all(vapply(d, function(x) all(tabulate(x + 1, 2) > 0), logical(1)))) {
        break
      }
    }
    return(list(data = d, misfit = rep(FALSE, 20)))
  }
  g <- power_grid[power_grid$cell == cell, ]
  row <- which(power_grid$cell == cell)
  d <- sim_power_data(n = g$n, n_misfit = g$n_misfit, target = g$target,
    direction = g$direction, rho = RH0, slope = SLOPE,
    seed = 1e6 * row + rep)
  list(data = d, misfit = seq_len(7) %in% item_setup(g$n_misfit, g$target)$misfit)
}

```

One run

One cutoff for one dataset, B and seed, scored with WY and BH, and with the two candidates from the same simulated values. `wy_local` recomputes the shipped WY with the arithmetic of `.bootstrap_pvalues()`, as a check that the candidates are computed on the same footing.

```

# Flags of the shipped |t| (local copy), |t| with pooled moments, and the
# pooled split test, from one dataset's observed differences and simulated

```

```

# null values. As split_variants() in restscore_power_conditional.qmd.
candidate_flags <- function(obs, sim, alpha = 0.05) {
  sim <- sim[, names(obs), drop = FALSE]
  B <- nrow(sim)
  stud <- function(ref) {
    m <- colMeans(ref, na.rm = TRUE)
    s <- apply(ref, 2L, stats::sd, na.rm = TRUE)
    s[!is.finite(s) | s < 1e-8] <- NA_real_
    list(o = (obs - m) / s, s = sweep(sweep(sim, 2L, m, `~`), 2L, s, `/`))
  }
  two <- function(z) {
    padj <- .wy_stepdown(abs(z$o), abs(z$s), B)
    ifelse(!is.na(padj) & padj < alpha,
           ifelse(z$o > 0, "overfit", "underfit"), "")
  }
  z_sim <- stud(sim)
  z_pool <- stud(rbind(obs, sim))
  hi <- .wy_stepdown(z_pool$o, z_pool$s, B)
  lo <- .wy_stepdown(-z_pool$o, -z_pool$s, B)
  data.frame(
    item = names(obs),
    wy_local = two(z_sim),
    wy_pooled = two(z_pool),
    split_pooled = ifelse(!is.na(hi) & hi < alpha / 2, "overfit",
                          ifelse(!is.na(lo) & lo < alpha / 2, "underfit", "")),
    stringsAsFactors = FALSE
  )
}

one_run <- function(task) {
  dd <- make_data(task$cell, task$rep)
  d <- dd$data
  seed <- 1e6 * task$rep + 1e3 * task$B + task$seed
  tryCatch({
    co <- suppressWarnings(suppressMessages(RMitemRestscoreCutoff(
      d, iterations = task$B, parallel = FALSE, seed = seed,
      cutoff_method = "quantile", dgp = "conditional"
    )))
    restscore <- function(correction) {
      suppressWarnings(suppressMessages(RMitemRestscore(
        d, cutoff = co, correction = correction, output = "dataframe"
      )))
    }
  }
  flags <- function(out) out$Flagged[match(names(d), out$Item)]
  fw <- restscore("fwer")
  sim_mat <- tapply(co$results$Difference,
                    list(co$results$iteration, co$results$Item),
                    function(x) x[1L])
  cand <- candidate_flags(stats::setNames(fw$Difference, fw$Item), sim_mat)
  cand <- cand[match(names(d), cand$item), ]
  data.frame(
    cell = task$cell, rep = task$rep, B = task$B, seed = task$seed,
    item = names(d), misfit = dd$misfit,
    wy = flags(fw), bh = flags(restscore("fdr_bh")),

```

```

    wy_pooled = cand$wy_pooled, split_pooled = cand$split_pooled,
    wy_local = cand$wy_local,
    iterations = co$actual_iterations,
    stringsAsFactors = FALSE
  )
}, error = function(e) {
  data.frame(cell = task$cell, rep = task$rep, B = task$B, seed = task$seed,
             error = conditionMessage(e))
})
}

```

Run

The runs within a cell are spread over the cores, the longest (B = 1000) first.

```

for (cl in cells) {
  f <- file.path(RES_DIR, paste0(cl, ".rds"))
  if (file.exists(f)) next
  tasks <- expand.grid(cell = cl, rep = seq_len(R_DATASETS),
                      seed = seq_len(N_SEEDS), B = sort(BS, decreasing = TRUE),
                      stringsAsFactors = FALSE)

  t0 <- Sys.time()
  res <- mclapply(seq_len(nrow(tasks)), function(i) one_run(tasks[i, ]),
                 mc.cores = NCORES, mc.preschedule = FALSE)
  saveRDS(list(res = res,
              elapsed_min = as.numeric(Sys.time() - t0, units = "mins")),
          f)
}

raw <- lapply(cells, function(cl) readRDS(file.path(RES_DIR, paste0(cl, ".rds"))))
all_rows <- do.call(rbind, lapply(raw, function(x) do.call(bind_rows, x$res)))
errors <- if ("error" %in% names(all_rows)) {
  filter(all_rows, !is.na(error))
} else {
  NULL
}
runs <- if (is.null(errors)) all_rows else filter(all_rows, is.na(error))
runs$cell <- factor(runs$cell, levels = cells)

data.frame(
  cell = cells,
  runs = vapply(cells, function(cl) {
    nrow(distinct(filter(runs, cell == cl), rep, B, seed))
  }, integer(1)),
  errors = vapply(cells, function(cl) {
    if (is.null(errors)) 0L else sum(errors$cell == cl)
  }, integer(1)),
  minutes = vapply(raw, function(x) round(x$elapsed_min, 1), numeric(1)),
  row.names = NULL
)

```

cell	runs	errors	minutes
underfit_1item_t0_n400	200	0	21.4

cell	runs	errors	minutes
underfit_2item_t0_n400	200	0	21.4
overfit_1item_t0_n150	200	0	17.2
overfit_1item_tm2_n400	200	0	21.4
dich_tgt_300	200	0	22.7

```
if (!is.null(errors)) count(errors, cell, error)

runs |>
  distinct(cell, rep, B, seed, iterations) |>
  group_by(cell, B) |>
  summarise(median_iterations = median(iterations),
            min_iterations = min(iterations), .groups = "drop")
```

cell	B	median_iterations	min_iterations
underfit_1item_t0_n400	400	400.0	400
underfit_1item_t0_n400	1000	1000.0	1000
underfit_2item_t0_n400	400	400.0	400
underfit_2item_t0_n400	1000	1000.0	1000
overfit_1item_t0_n150	400	397.0	344
overfit_1item_t0_n150	1000	993.5	875
overfit_1item_tm2_n400	400	400.0	395
overfit_1item_tm2_n400	1000	1000.0	989
dich_tgt_300	400	400.0	400
dich_tgt_300	1000	1000.0	1000

```
# The local copy must reproduce the package's WY flags in every run.
sum(runs$wy_local != runs$wy)
```

```
[1] 0
```

```
methods <- c(WY = "wy", BH = "bh", WY_pooled = "wy_pooled",
             split_pooled = "split_pooled")
```

Results

Two-analyst disagreement

For each dataset, every pair of seeds at the same B is compared. A pair disagrees when at least one item gets a different flag.

```
pair_disagreement <- function(runs, method) {
  runs |>
    select(cell, rep, B, seed, item, flag = all_of(method)) |>
    group_by(cell, rep, B) |>
```

```

group_modify(function(d, key) {
  w <- tidyr::pivot_wider(d, names_from = seed, values_from = flag)
  s <- setdiff(names(w), "item")
  pr <- utils::combn(s, 2)
  data.frame(disagree = apply(pr, 2, function(p) {
    any(w[[p[1]]] != w[[p[2]]])
  }))
}) |>
ungroup()
}

dis <- bind_rows(lapply(names(methods), function(m) {
  mutate(pair_disagreement(runs, methods[[m]]), method = m)
})) |>
mutate(method = factor(method, levels = names(methods)))
dis |>
group_by(cell, method, B) |>
summarise(pairs = n(), disagree = round(100 * mean(disagree), 1),
  .groups = "drop") |>
pivot_wider(names_from = B, values_from = c(pairs, disagree),
  names_glue = "{.value}_B{B}")

```

cell	method	pairs_B400	pairs_B1000	disagree_B400	disagree_B1000
underfit_1item_t0_n400	WY	200	200	0.0	0.0
underfit_1item_t0_n400	BH	200	200	12.0	11.0
underfit_1item_t0_n400	WY_pooled	200	200	0.0	0.0
underfit_1item_t0_n400	split_pooled	200	200	0.0	0.0
underfit_2item_t0_n400	WY	200	200	22.0	12.0
underfit_2item_t0_n400	BH	200	200	21.0	22.0
underfit_2item_t0_n400	WY_pooled	200	200	20.5	13.0
underfit_2item_t0_n400	split_pooled	200	200	20.5	13.0
overfit_1item_t0_n150	WY	200	200	15.0	7.5
overfit_1item_t0_n150	BH	200	200	14.5	13.0
overfit_1item_t0_n150	WY_pooled	200	200	14.0	7.5
overfit_1item_t0_n150	split_pooled	200	200	16.0	8.0
overfit_1item_tm2_n400	WY	200	200	7.0	6.0
overfit_1item_tm2_n400	BH	200	200	14.0	5.0
overfit_1item_tm2_n400	WY_pooled	200	200	9.0	6.0
overfit_1item_tm2_n400	split_pooled	200	200	8.0	2.0
dich_tgt_300	WY	200	200	6.0	5.0
dich_tgt_300	BH	200	200	6.0	6.0
dich_tgt_300	WY_pooled	200	200	6.0	5.0
dich_tgt_300	split_pooled	200	200	7.0	0.0

```
dis |>
  group_by(method, B) |>
  summarise(pairs = n(), disagree = round(100 * mean(disagree), 1),
            .groups = "drop")
```

method	B	pairs	disagree
WY	400	1000	10.0
WY	1000	1000	6.1
BH	400	1000	13.5
BH	1000	1000	11.4
WY_pooled	400	1000	9.9
WY_pooled	1000	1000	6.3
split_pooled	400	1000	10.3
split_pooled	1000	1000	4.6

Unstable items

Items whose flag is not the same in all five seeds.

```
runs |>
  pivot_longer(all_of(unname(methods)), names_to = "method",
               values_to = "flag") |>
  mutate(method = factor(method, levels = methods, labels = names(methods))) |>
  group_by(cell, method, B, rep, item) |>
  summarise(unanimous = n_distinct(flag) == 1, .groups = "drop") |>
  group_by(cell, method, B) |>
  summarise(unstable_items = round(100 * mean(!unanimous), 1),
            .groups = "drop") |>
  pivot_wider(names_from = B, values_from = unstable_items,
              names_prefix = "B")
```

cell	method	B400	B1000
underfit_1item_t0_n400	WY	0.0	0.0
underfit_1item_t0_n400	BH	3.6	2.9
underfit_1item_t0_n400	WY_pooled	0.0	0.0
underfit_1item_t0_n400	split_pooled	0.0	0.0
underfit_2item_t0_n400	WY	7.1	3.6
underfit_2item_t0_n400	BH	7.1	6.4
underfit_2item_t0_n400	WY_pooled	7.1	3.6
underfit_2item_t0_n400	split_pooled	6.4	3.6
overfit_1item_t0_n150	WY	4.3	2.9
overfit_1item_t0_n150	BH	6.4	5.0
overfit_1item_t0_n150	WY_pooled	4.3	2.9
overfit_1item_t0_n150	split_pooled	5.0	2.1
overfit_1item_tm2_n400	WY	2.1	1.4

cell	method	B400	B1000
overfit_1item_tm2_n400	BH	4.3	1.4
overfit_1item_tm2_n400	WY_pooled	2.1	1.4
overfit_1item_tm2_n400	split_pooled	2.1	0.7
dich_tgt_300	WY	0.5	0.5
dich_tgt_300	BH	0.5	0.5
dich_tgt_300	WY_pooled	0.5	0.5
dich_tgt_300	split_pooled	0.8	0.0

Flags per dataset

Average number of items flagged per run, misfitting items in the correct direction and other flags separately. Under the null cell every flag is false.

```
runs |>
  left_join(
    tibble::tibble(cell = cells,
                   direction = c("underfit", "underfit", "overfit",
                                "overfit", "none")),
    by = "cell"
  ) |>
  pivot_longer(all_of(unname(methods)), names_to = "method",
               values_to = "flag") |>
  mutate(method = factor(method, levels = methods, labels = names(methods))) |>
  group_by(cell, method, B, rep, seed) |>
  summarise(correct = sum(misfit & flag == direction),
            other = sum(flag != "" & !(misfit & flag == direction)),
            .groups = "drop") |>
  group_by(cell, method, B) |>
  summarise(correct = round(mean(correct), 2), other = round(mean(other), 2),
            .groups = "drop") |>
  arrange(cell, method, B)
```

cell	method	B	correct	other
dich_tgt_300	WY	400	0.00	0.06
dich_tgt_300	WY	1000	0.00	0.04
dich_tgt_300	BH	400	0.00	0.04
dich_tgt_300	BH	1000	0.00	0.04
dich_tgt_300	WY_pooled	400	0.00	0.06
dich_tgt_300	WY_pooled	1000	0.00	0.04
dich_tgt_300	split_pooled	400	0.00	0.04
dich_tgt_300	split_pooled	1000	0.00	0.00
overfit_1item_t0_n150	WY	400	0.27	0.14
overfit_1item_t0_n150	WY	1000	0.22	0.13
overfit_1item_t0_n150	BH	400	0.25	0.14
overfit_1item_t0_n150	BH	1000	0.29	0.16

cell	method	B	correct	other
overfit_1item_t0_n150	WY_pooled	400	0.26	0.14
overfit_1item_t0_n150	WY_pooled	1000	0.22	0.13
overfit_1item_t0_n150	split_pooled	400	0.29	0.12
overfit_1item_t0_n150	split_pooled	1000	0.23	0.12
overfit_1item_tm2_n400	WY	400	0.64	0.12
overfit_1item_tm2_n400	WY	1000	0.62	0.08
overfit_1item_tm2_n400	BH	400	0.58	0.15
overfit_1item_tm2_n400	BH	1000	0.63	0.15
overfit_1item_tm2_n400	WY_pooled	400	0.62	0.09
overfit_1item_tm2_n400	WY_pooled	1000	0.62	0.07
overfit_1item_tm2_n400	split_pooled	400	0.63	0.08
overfit_1item_tm2_n400	split_pooled	1000	0.64	0.05
underfit_1item_t0_n400	WY	400	1.00	0.10
underfit_1item_t0_n400	WY	1000	1.00	0.10
underfit_1item_t0_n400	BH	400	1.00	0.17
underfit_1item_t0_n400	BH	1000	1.00	0.18
underfit_1item_t0_n400	WY_pooled	400	1.00	0.10
underfit_1item_t0_n400	WY_pooled	1000	1.00	0.10
underfit_1item_t0_n400	split_pooled	400	1.00	0.10
underfit_1item_t0_n400	split_pooled	1000	1.00	0.10
underfit_2item_t0_n400	WY	400	0.50	0.23
underfit_2item_t0_n400	WY	1000	0.51	0.21
underfit_2item_t0_n400	BH	400	0.48	0.27
underfit_2item_t0_n400	BH	1000	0.52	0.33
underfit_2item_t0_n400	WY_pooled	400	0.50	0.19
underfit_2item_t0_n400	WY_pooled	1000	0.51	0.20
underfit_2item_t0_n400	split_pooled	400	0.45	0.20
underfit_2item_t0_n400	split_pooled	1000	0.46	0.19

Interpretation

All 1,000 runs ran without errors (17 to 23 minutes per cell), and the local copy of the WY arithmetic reproduced the package’s flags in every run, so the candidates were scored on the same simulated values as the shipped test.

The results rest on 20 datasets per cell, and the 10 seed pairs within a dataset are not independent, so differences of a few points between methods at the same B are within noise. The drop from B = 400 to B = 1000 is consistent across methods and cells.

1. At B = 400, about one analysis in ten depends on the seed. Over all cells, two seeds disagreed on the flag of at least one item in 10.0 percent of pairs with WY, close to the 10 percent found for infit at B = 400 (Johansson, 2026). The disagreement is concentrated where decisions are hard: 22 percent

with two underfitting items masking each other and 15 percent with overfit at $n = 150$, against 0 with one clear underfitting item at $n = 400$.

2. $B = 1000$ reduces it to about 6 percent. WY disagreement fell from 10.0 to 6.1 percent and the share of unstable items roughly halved in the hard cells (7.1 to 3.6 percent with two underfitting items). B did not shift decisions in one direction: correct and other flags per dataset were nearly the same at both values for every method.

3. The split test is as stable as $|t|$. Its critical value is a more extreme quantile of the simulated maxima, which could have made it more seed-dependent at $B = 400$. It did not: `split_pooled` disagreed in 10.3 percent of pairs against 10.0 for WY, with 0 to 20.5 percent by cell against 0 to 22, and 0 to 6.4 percent unstable items against 0 to 7.1. At $B = 1000$ it was the most stable method, 4.6 percent overall and 0 in the null cell `dich_tgt_300`. Pooled moments did not change stability (`WY_pooled` 9.9 and 6.3). With two underfitting items `split_pooled` detected fewer per run (0.45 against 0.50), the underfit cost seen in `restscore_power_conditional.qmd`.

4. BH is the least stable and gains little from more iterations. BH disagreed in 13.5 percent of pairs at $B = 400$ and 11.4 at $B = 1000$, more than WY at both. Its discrete p-values and the α/k threshold for the smallest one leave it sensitive to where the observed value falls among the simulated ones even at $B = 1000$.

Implications. Seed stability does not count against the split test, so the choice between it and $|t|$ rests on the trade measured in the power study. Separately, the halving of seed dependence at $B = 1000$ bears on the default number of iterations, which is 400 now.